Nonhalting and PA-independence for a 196-state Turing machine

James Chang

Abstract

We give a two-symbol Turing machine with 196 working states that searches for a contradiction in ordinary first-order Peano arithmetic. It runs forever on a blank tape. Peano arithmetic proves neither its exact arithmetic halting sentence nor its negation. The machine's axiom command reverses the two arguments in the quantifier code and uses a guarded equality-vacuity clause in place of a primitive transitivity head. In Lean 4 we verify the calculus, the simulation of the register program, and both unprovability results for the literal table.

1 The result

Let M be the machine in Section 2, and let M_{209} be its predecessor [1]. Write PA for ordinary first-order Peano arithmetic. The arithmetic halting sentence H uses the rows of M in the trace formula of [1, §5].

Theorem 1.1. *If* PA *is consistent, the machine M does not halt from the blank tape and* $\mathrm{PA} \nvdash \neg H$. *If* PA *is sound for the standard model, also* $\mathrm{PA} \nvdash H$.

The change to the register program is confined to the body of one command (Lemma 3.1). We prove soundness for the revised calculus in Section 4 and show how the compiler of [1] sends PA refutations to proofs in that calculus in Section 5. Sections 6 and 7 give the tape layout and its simulation proof.

Lean uses Foundation's standard model of ordinary PA to establish consistency and soundness in the metatheory. This yields closed Lean theorems for blank-tape nonhalting and both unprovability directions.

2 The machine

The tape has alphabet $\{0, 1\}$ and is infinite in both directions. It starts with all cells zero. State 0 is the start state, $0, \ldots, 195$ are the working states, and 196 is the halt label. In state q, a row (q, b, w, D, q') reads b, writes w, moves one cell in direction D, and enters q'. The table has one row for each pair in $\{0, \ldots, 195\} \times \{0, 1\}$. The file `data/pa196.tm` contains all 392 rows and has SHA-256 digest

`e8bbccd4cfa325251ca95066db116b5aa766e404b0971c33c02772be1c3617a8`.

The first row is $(0, 0, 1, R, 1)$, and $(63, 1, 1, R, 196)$ is the only row that enters the halt label.

As in M_{209}, a tape engine interprets a register program stored as a binary tree. A *frame* records a marker, the path to the current site, a gap, and the register file. A block $0\,1^{v+1}$ stores value

v [1, §3]. The engine's 36 states have the same rows as in M_{209} after renaming. Of the remaining states, 13 route leaf instructions to the engine and 147 read the 159 distinct nodes. State 75 reads the root.

3 The register program

The register program of M_{209} enumerates words over three commands on a store (t, w, d, h). It halts when Shift reads formula code 1 [1, §4]. We keep Shift and MP, change $\mathsf{AX}(c, x, y)$ in three steps, and then change how the program stores that command.

3.1 Step 1. The quantifier code

Let $p(a, b) = \binom{a+b+1}{2} + b$. The four constructors in M_{209} use $4p(a, b) + r$, where $r = 0, 1, 2, 3$ denotes implication, membership, universal quantification and equality, respectively. The constructors in M are

$$\mathsf{I}(a, b) = 4p(a, b), \quad \mathsf{M}(a, b) = 4p(a, b) + 1, \quad \mathsf{U}(a, B) = 4p(B, a) + 2, \quad \mathsf{E}(a, b) = 4p(a, b) + 3.$$

Only the quantifier code reverses the pairing arguments. As before, code 0 is truth and $1 = \mathsf{M}(0, 0)$ is the contradiction. Write $\mathsf{N}(B) = \mathsf{I}(B, 1)$, $B \wedge C = \mathsf{N}(\mathsf{I}(B, \mathsf{N}(C)))$, $B \leftrightarrow C$ for $\mathsf{I}(B, C) \wedge \mathsf{I}(C, B)$, and $\exists x_a\, B$ for $\mathsf{N}(\mathsf{U}(a, \mathsf{N}(B)))$. An arrow in the formulas below denotes I.

3.2 Steps 2 and 3. The axiom heads

In M_{209}, AX pushes the saved top, a generalization, the restricted propositional head ρ_8 and seven further heads $\rho_1, \ldots, \rho_7$. It leaves a guarded gate in t. For $\mathsf{AX}(c, x, y)$ in M, put $D = d + x$, $H = h + y$ and $A = c$. The command pushes the saved top t, the generalization $\mathsf{U}(d, h)$, the head

$$\rho_8(D, A, H, d) = \big((H \to d) \to D\big) \to \big((D \to H) \to (A \to H)\big),$$

and then the six formulas of Table 1, in that order.

Push	Formula	Head of [1]
1	$(\exists x_D\, \mathsf{U}(D, H)) \to H$	ρ_6
2	$\mathsf{U}(D, \mathsf{U}(A, H)) \to \mathsf{U}(A, \mathsf{U}(D, H))$	ρ_4
3	$\mathsf{M}(H, D) \to (\mathsf{E}(H, A) \to \mathsf{M}(A, D))$	ρ_5, antecedents exchanged
4	$\mathsf{E}(H, D) \to (\mathsf{M}(A, H) \to \mathsf{M}(A, D))$	ρ_3
5	$\exists x_D\, \mathsf{E}(H, D)$	ρ_1, equation mirrored
6	$\mathsf{U}(D, A) \to (\mathsf{U}(D, A \to H) \to \mathsf{U}(D, H))$	ρ_2, antecedents exchanged

Table 1: The six heads pushed by AX after the saved top, $\mathsf{U}(d, h)$ and ρ_8.

The command leaves 0 in t if $A = H$ or $A = D$, and $F(H, D, A)$ otherwise. For labels a, b and a code c, the gate F is defined by

$$\Delta_c(a, b) = \mathsf{E}(a, b) \to \mathsf{U}(c, \mathsf{E}(a, b)), \qquad V = \mathsf{M}(a, b) \to \mathsf{U}(c, \mathsf{M}(a, b)),$$
$$P = c \to \mathsf{M}(c, b), \qquad S = \mathsf{U}\big(b,\ (\mathsf{M}(a, b) \to P) \wedge (\mathsf{M}(c, b) \to c)\big),$$
$$Q = \mathsf{U}(c,\ S \to \mathsf{N}(P)), \qquad F(a, b, c) = \mathsf{N}\big(\Delta_c(a, b) \to (V \to Q)\big).$$

The command finishes by setting $d = h = 0$. It pushes nine items in all, including the saved top. In P, S and Q, the number c is both the label of x_c and a formula code.

Under the guard $c \neq a, b$, the gate $G(a, b, c) = \mathsf{N}(V \to Q)$ of [1] gives membership vacuity V and pointed separation $\mathsf{N}(Q)$. Step 2 removes the seventh head

$$\rho_7(a, b, c) = \mathsf{E}(c, a) \to (\mathsf{E}(c, b) \to \mathsf{U}(\mathsf{M}(a, b), \mathsf{E}(a, b)))$$

and adds equality vacuity $\Delta_c(a, b)$ to the gate under the same guard. Step 3 rearranges the heads as shown in the third column of Table 1 and exchanges the second and third arguments of ρ_8.

The body of AX is a straight-line sequence of 201 items. Its macro calls come from [1]. They push the accumulator, push it and load one of the three registers, pop and form $4p$, read the two parameters, or check the two parts of the guard. The other items are single increments of the accumulator and the closing jump. The full program has 3,812 sites, compared with 3,948 for M_{209}.

Lemma 3.1 (Program)**.** *Outside the body of* AX *the register program R of M agrees with that of M_{209} after translation of addresses. The body of* AX *is a concatenation of blocks, each identical to a macro block of M_{209} and left only to the next block. It computes the saved top, $\mathsf{U}(d, h)$, $\rho_8(D, A, H, d)$, the six formulas of Table 1 and the gate $F(H, D, A)$, in the codes of Step 1.*

Proof. After flattening the program and translating the addresses, the 169 sites outside the body agree with M_{209}. The 201 blocks use the same macros and internal jump offsets. The closing jump goes to the corresponding successor. The macro contracts give the store effect of each block. Symbolic execution gives the nine stack entries and the new top as formula trees over $4p(\cdot, \cdot) + r$, with the argument order of Step 1. Lean checks the source program in `PA196/Program.lean` and the effect of the command in `PA196/Source/AXEffect.lean`. □

4 Soundness

Lemma 4.1. *If* PA *is consistent, R does not halt from the clean store.*

Proof. Use the interpretation of membership and equality from [1, §4.2]. In a model of PA, $\mathsf{M}(a, b)$ says that bit x_b of x_a is set. Each formula in Table 1 is either a head of [1], possibly with two antecedents exchanged, or the witness formula $\exists x_a(x_b = x_a)$. Each is true under every assignment, as is the propositional tautology ρ_8.

When $c \neq a, b$, $G(a, b, c)$ is true by [1]. The atom $\mathsf{E}(a, b)$ contains no x_c, so $\Delta_c(a, b)$ is also true. Thus $F(a, b, c)$ is true, since it is propositionally $G \wedge \Delta_c$. If the guard fails, the gate returns truth. Modus ponens and generalization preserve truth under every assignment, and code 1 is false. We use the invariant, halt test and scheduler of [1, §4.2].

The encoder and decoder exchange the quantifier arguments together. For a nonzero code $4p(u, v) + r$, both components are smaller than the code in either order. The decoder still recurses on smaller codes, and every label in the formula with code c is below c. Thus x_c is fresh in the gate. □

5 Completeness

A formula is *derivable* if modus ponens and generalization give a proof from instances of the six heads, ρ_8 with $u \leq a$, and the gate with $c \neq a, b$. The proofs below use fixed finite templates in the labels and codes. Their case distinctions compare numbers for equality. PA can carry out these arguments on coded proofs by the method of [1, §6.1].

5.1 Uses of the seventh head

In the Lean development of [1], only `ActualCertificateNeighborhoodPrimitive.lean` uses ρ_7 directly. It derives reflexivity and symmetry of primitive equality and vacuity of $\mathsf{E}(a,b)$ under the label $\mathsf{M}(a,b)$. `ActualCertificateNeighborhoodCongruence.lean` uses symmetry to reverse the membership rewrites ρ_3, ρ_5. `ActualCertificateNeighborhoodWitness.lean` uses symmetry and vacuity to prove $\mathsf{E}(a,b) \to \mathsf{R}(a,b,z)$ for $z \neq a,b$, where

$$\mathsf{R}(a,b,z) = \mathsf{U}\big(z, \mathsf{M}(a,z) \leftrightarrow \mathsf{M}(b,z)\big) \wedge \mathsf{U}\big(z, \mathsf{M}(z,a) \leftrightarrow \mathsf{M}(z,b)\big)$$

is the equality relation of the interpretation. The same module then gets $\exists x_a\, \mathsf{R}(a,c,z)$ from ρ_1. The rest of the proof uses primitive equality through the following three facts.

(R) For all labels a, b, d, both $\mathsf{E}(a,b) \to (\mathsf{M}(d,a) \leftrightarrow \mathsf{M}(d,b))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(a,d) \leftrightarrow \mathsf{M}(b,d))$.

(N) $\mathsf{E}(a,b) \to \mathsf{R}(a,b,z)$ for $z \neq a,b$.

(W) An existential witness for equality at every pair of labels.

We derive (R), (N) and (W) directly. They suffice for the interpretation in place of separate laws for symmetry and transitivity of primitive equality.

5.2 Proof rules used below

1. The classical implicational rules come from ρ_8 as in [1]. Use code 1 for falsity. Head 1 with $X = \mathsf{U}(D, \mathsf{N}(\mathsf{U}(D,H)))$ gives $\mathsf{N}(X) \to H$. Implicational reasoning gives $1 \to (X \to 1)$, so $1 \to H$ for every formula H. Double negation elimination follows.

2. To lift a derivable implication $B \to C$ to $\mathsf{U}(x,B) \to \mathsf{U}(x,C)$, use generalization and head 6.

3. A formula B is vacuous at r if $B \to \mathsf{U}(r,B)$ is derivable. The gate gives vacuity for both $\mathsf{M}(a,b)$ and $\mathsf{E}(a,b)$ whenever $r \neq a,b$. If B is vacuous at r, lift $\mathsf{N}(\mathsf{U}(r,B)) \to \mathsf{N}(B)$ at r and use head 1 with $D = r$ to get $\mathsf{N}(\mathsf{U}(r,\mathsf{N}(B))) \to B$. Thus $\mathsf{N}(B)$ is vacuous at r. Vacuity passes to implications by lifting $\mathsf{N}(B) \to (B \to C)$ and $C \to (B \to C)$. It passes to $\mathsf{U}(x,B)$ when $x \neq r$ by lifting at x and using head 2. A formula built from atoms is therefore vacuous at every label outside its atom indices and bound labels.

4. If B is vacuous at x, then $\mathsf{U}(x, A \to B)$ and $\exists x\, A$ give B. If $\mathsf{N}(B)$ holds, vacuity gives $\mathsf{U}(x, \mathsf{N}(B))$, which yields $\mathsf{U}(x, \mathsf{N}(A))$.

Rules 1, 2 and 4 are from [1]. Rule 3 extends the vacuity argument of [1] to equality atoms.

5.3 The gate

Lemma 5.1. *$F(a,b,c) \to G(a,b,c)$ and $F(a,b,c) \to \Delta_c(a,b)$ are derivable.*

Proof. Put $B = (V \to Q)$, so $F = \mathsf{N}(\Delta_c \to B)$ and $G = \mathsf{N}(B)$. Weakening B gives $\Delta_c \to B$, so $F \to \mathsf{N}(B)$. The assumption $\mathsf{N}(\Delta_c)$ also gives $\Delta_c \to B$, by explosion. Therefore $F \to \mathsf{N}(\mathsf{N}(\Delta_c))$, and double negation elimination gives Δ_c under F. □

The new gate thus gives the old gate at the same parameters. For a label u and a formula φ with code $c \neq u$, the instance at (u,u,c) gives separation in the form

$$\exists x_c\, (S \wedge P), \qquad P = \varphi \to \mathsf{M}(c,u), \qquad S \to \mathsf{U}\big(u,\ \mathsf{M}(c,u) \to \varphi\big). \tag{1}$$

The variable x_u is bound in S and free in P.

5.4 Reverse rewriting

Heads 4 and 3 give the direct rewrites $\mathsf{E}(a,b) \to (\mathsf{M}(d,a) \to \mathsf{M}(d,b))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(a,d) \to \mathsf{M}(b,d))$ for all labels. Let $\chi(t)$ be one of the atoms $\mathsf{M}(d,t)$, $\mathsf{M}(t,d)$ and $\mathsf{M}(t,t)$, where d is a label. The direct rewrites give

$$\mathsf{E}(v,y) \to (\chi(v) \to \chi(y)) \tag{2}$$

for all labels v and y. For $\mathsf{M}(t,t)$, rewrite one position at a time.

Lemma 5.2 (Specialization of a negative property)**.** *Let y, v, c be labels with $v \neq y$ and $c \neq y$, and let χ be as above with $d \neq y$. Then*

$$\mathsf{U}(y,\ \mathsf{M}(c,y) \to \mathsf{N}(\chi(y)))\ \ \to\ \ (\mathsf{M}(c,v) \to \mathsf{N}(\chi(v)))$$

is derivable.

Proof. Write $\psi(t) = \mathsf{M}(c,t) \to \mathsf{N}(\chi(t))$. Head 4 gives $\mathsf{E}(v,y) \to (\mathsf{M}(c,v) \to \mathsf{M}(c,y))$, and (2) gives $\mathsf{E}(v,y) \to (\chi(v) \to \chi(y))$. Heads 3 and 4 allow $d = v$ because they have no side condition. These direct rewrites suffice for $\mathsf{E}(v,y) \to (\psi(y) \to \psi(v))$, since y occurs in $\psi(y)$ only in an antecedent and under a negation. Lift at y to obtain $\mathsf{U}(y,\psi(y)) \to \mathsf{U}(y,\mathsf{E}(v,y) \to \psi(v))$. The indices of $\psi(v)$ lie among c, v and d, all different from y, so $\psi(v)$ is vacuous at y. Apply existential elimination with the instance $\exists x_y\, \mathsf{E}(v,y)$ of head 5 to get $\psi(v)$. □

Lemma 5.3 (Reverse rewriting)**.** *For all labels a, b, d, $\mathsf{E}(a,b) \to (\mathsf{M}(d,b) \to \mathsf{M}(d,a))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(b,d) \to \mathsf{M}(a,d))$ are derivable.*

Proof. For $a = b$ both are instances of $B \to (C \to C)$. Let $a \neq b$, and let χ be as above with $d \neq a$. We first derive

$$\mathsf{E}(a,b) \to (\chi(b) \to \chi(a)). \tag{3}$$

Let $\varphi = \mathsf{N}(\chi(a))$. If its code equals b, take $\mathsf{N}(\mathsf{N}(\mathsf{N}(\chi(a))))$ instead, whose code is larger. Let c be the code of φ. Then c exceeds a and d and differs from b. Take the separation instance (1) with $u = a$. By Lemma 5.2 with $y = a$ and $v = b$,

$$S \to (\mathsf{M}(c,b) \to \mathsf{N}(\chi(b))),$$

and head 4 gives $\mathsf{E}(a,b) \to (\mathsf{M}(c,a) \to \mathsf{M}(c,b))$. Hence $(S \wedge P) \to C$ is derivable for the formula C in (3). Suppose $\mathsf{E}(a,b)$ and $\chi(b)$ hold and $\chi(a)$ fails. Then φ holds, so P gives $\mathsf{M}(c,a)$. The rewrite gives $\mathsf{M}(c,b)$, and S gives $\mathsf{N}(\chi(b))$. The indices of C are a, b and d, all different from c. Vacuity for the equality atom makes C vacuous at c, so existential elimination with (1) gives C.

For $d \neq a$, (3) with $\chi(t) = \mathsf{M}(d,t)$ and with $\chi(t) = \mathsf{M}(t,d)$ are the two statements. For $d = a$ the statements are $\mathsf{E}(a,b) \to (\mathsf{M}(a,b) \to \mathsf{M}(a,a))$ and $\mathsf{E}(a,b) \to (\mathsf{M}(b,a) \to \mathsf{M}(a,a))$. Head 3 gives $\mathsf{E}(a,b) \to (\mathsf{M}(a,b) \to \mathsf{M}(b,b))$, head 4 gives $\mathsf{E}(a,b) \to (\mathsf{M}(b,a) \to \mathsf{M}(b,b))$, and (3) with $\chi(t) = \mathsf{M}(t,t)$ gives $\mathsf{E}(a,b) \to (\mathsf{M}(b,b) \to \mathsf{M}(a,a))$. □

In the interpretation, separation gives a set x_c that contains x_a if x_a lacks χ and contains only elements that lack χ. Since x_b belongs to every set that contains x_a, if x_b has χ, then x_a has it too. The calculus specializes a quantifier with a witness from head 5 and direct rewrites. These steps supply the specialization without a substitution rule.

Lemma 5.4 (The defined equality)**.** *For $z \neq a, b$, $\mathsf{E}(a, b) \to \mathsf{R}(a, b, z)$ is derivable.*

Proof. The direct rewrites and Lemma 5.3 with $d = z$ give $\mathsf{E}(a, b) \to (\mathsf{M}(a, z) \leftrightarrow \mathsf{M}(b, z))$ and $\mathsf{E}(a, b) \to (\mathsf{M}(z, a) \leftrightarrow \mathsf{M}(z, b))$. Lift both at z and compose with $\mathsf{E}(a, b) \to \mathsf{U}(z, \mathsf{E}(a, b))$ from the gate. □

The Lean proofs of reverse rewriting and neighborhood equality are in `PA196/Logic/Reverse.lean` and `PA196/Logic/Neighborhood.lean`. The certificate compiler builds the same derivations inside arbitrary PA models.

5.5 Transporting PA refutations

Lemma 5.5 (Completeness)**.** PA *proves that if* $\mathrm{Prov}_{\mathrm{PA}}(\ulcorner \bot \urcorner)$ *then R halts from the clean store.*

Proof. The direct rewrites and Lemma 5.3 give (R), Lemma 5.4 gives (N), and head 5 gives (W) with the equation reversed. To specialize a quantifier at z to a label c, the proof in [1] assumes $\mathsf{E}(z, c)$ and rewrites atoms in both directions. With the head 5 witness, the assumption is $\mathsf{E}(c, z)$, and (R) still gives both rewrites. For the witness of the defined equality, apply existential monotonicity to $\exists x_a\, \mathsf{E}(c, a)$ using (N) to get $\exists x_a\, \mathsf{R}(c, a, z)$. The definition of R gives its symmetry, and hence $\exists x_a\, \mathsf{R}(a, c, z)$.

Lemma 5.1 gives the old gate. Heads 1, 2, 4 and ρ_8 give the old heads ρ_6, ρ_4, ρ_3 and ρ_8. Exchanging two antecedents in heads 3 and 6 gives ρ_5 and ρ_2 by propositional reasoning. With these derivations in place of the ones that use ρ_7, the construction of [1, §6.1] turns a coded PA refutation into a coded derivation of code 1 in the present calculus.

Step 1 preserves the four coding properties the construction uses. The encoder and decoder are inverse, both components are smaller than their compound code, the residues modulo 4 are the same, and MP still computes $\mathsf{I}(t, y) = 4p(t, y)$. When we choose a formula code as a label, we use the new constructor. The code of φ in Lemma 5.3 and the padded property code for separation satisfy the same bounds.

The nine items pushed by AX change the command identities of [1, §6.2]. Writing S for Shift, the first is

$$\mathsf{AX}(c, x, y)\, \mathsf{S}^9(t, w, d, h) = (\mathsf{U}(d, h), w, 0, t).$$

The following words implement the three stack operations.

$$\begin{aligned} \text{pop} \quad & \mathsf{AX}(0, 0, 0)\, \mathsf{S}^{10}, \\ \text{push} \quad & \mathsf{AX}(0, 0, 0)\, \mathsf{S}^8\, \mathsf{MP}(0), \\ \text{discard} \quad & \mathsf{MP}(1)\, \mathsf{AX}(1, 0, 0)\, \mathsf{S}^9\, \mathsf{MP}(0). \end{aligned}$$

Each formula these Shifts test is 0 or at least 2. The heads and gate have residue 0, and $\mathsf{U}(d, h)$ has residue 2. To select head j from the top, for $1 \leq j \leq 7$, discard, call AX, then apply $j + 1$ Shifts and $8 - j$ pops. To select the gate, discard, call AX, then pop nine times. Discard sets $d = h = 0$, so $(D, H, A) = (x, y, c)$. Read the corresponding row of Table 1 backwards to choose the arguments for an instance $\rho(\alpha, \beta, \gamma)$. For $\rho_8(a, b, c, u)$ with $u \leq a$, set the offset to u and call $\mathsf{AX}(b, a - u, c)$.

The generalization word pushes, applies offset i and a Shift, calls $\mathsf{AX}(i, 0, 0)$ and applies S^9. The modus ponens suffix is $\mathsf{S}^2\, \mathsf{MP}(b)\, \mathsf{AX}(b, 0, 0)\, \mathsf{S}^{10}$. With these words, the induction on fuel and the scheduler argument of [1, §6.2–6.3] apply as written. □

6 The layout

At a node, the state reads the path bit, moves right and enters the state for the corresponding child. Leaves are instructions, and the machine stores equal subtrees once. The 201 items in the body of AX form the leaves of a binary tree with 68 distinct internal nodes.

As in M_{209}, two nodes can share a state if one occurs only as a left child and the other only as a right child. When the current bit would select different children for the two nodes, the shared state moves left to the bit that selected the node. A second state reads that bit again and moves right into a third state, which uses the current bit to choose the correct child. Both states already read other nodes or start a macro, so sharing requires no extra states. The six pairs in Table 2 let 62 states read the 68 nodes.

State	Left-child node	Right-child node	Rows sent left
138	(D | HC)	(PiCDCiiPiC | HC)	$0 \to 168$
140	(C | C)	(H | C)	$0 \to 158$
160	(i | ADCi)	(i | C)	$1 \to 185$
174	(i | ACiiCHDCi)	(DCiiHDCii | ACiiCHDCi)	$0 \to 172$
179	(A | ADCi)	(CC | ADCi)	$0 \to 142$
183	(C | PiC)	(H | DCi)	$0 \to 158$, $1 \to 167$

Table 2: States of M that read two nodes of the body of AX. Each node lists its two children as strings of items. P pushes. D, H and A push and load a register. C pops and forms $4p$, and i adds one. The last column gives each symbol whose row moves left and the state it enters.

We remove two states outside the body that M_{209} used to distinguish nodes of its old body. Each read a single symbol and stepped back onto the cell just written. The row that entered it now performs the successor's action directly.

7 Simulation

Number the cells of a frame from the first path cell. In a *clean* frame, zeros separate the path from the header 1, and the register file follows the header. The *gap* is the position of that header. The longest path in R is 38 bits. The formal simulation uses reachable frames with a gap of at least 48 and a file of at least ten registers. The clean startup frame meets the gap bound, and the simulation preserves it.

Lemma 7.1 (One instruction)**.** *For every site of R and each outcome of a conditional decrement, the machine M started in state* 75 *on a valid clean frame of that site with gap at least 48 reaches, in at least one step, the clean frame of the successor site with the register effect of the instruction. At the halt site it enters the halt label.*

Proof. A run from a clean frame alternates between walks outside the engine and calls to the engine. The engine rows match those of M_{209}. The register lemmas of [1, §3] apply for the same register contents and gaps. They cover filling the gap, marking a register, incrementing, both outcomes of decrementing, updating the path and scanning back. The return scan ends on the cell after the first 1 with seven zeros to its left. For most sites that cell is the root. At 19 sites one pass ends on a later path cell, where the next walk begins, as in M_{209}.

In the computed runs, states outside the engine read only path cells and the two cells after the path. These reads stay at or to the right of the root. At each site and for each outcome, the walk

visits the same sequence of states, cells and symbols for every register file tested. One run thus determines the fixed sequence of rows and engine calls for the contract.

We ran the checks at all 3,812 sites, including both outcomes at each of the 1,147 decrement sites. We varied the register contents and used gaps 40, 41, 48 and 61. All 68,616 runs end in the required frame, and together they read all 392 rows. □

The coverage theorem in `PA196/Coverage.lean` checks the positive-length contract at every site for the reachable gap bound, including both outcomes of a conditional decrement.

Lemma 7.2 (Startup)**.** *From the blank tape M reaches after exactly* 58,154 *steps the clean frame of site* 25 *with gap* 48*, candidate register* 2*, every other logical register* 0*, and* 26 *further registers equal to* 1*.*

Proof. After running M for 58,154 steps, the tape matches the startup frame of M_{209} cell for cell [1, §2]. The head is on the first path cell in state 75. Lean checks each segment and joins them to obtain the literal startup trace. □

The scheduler starts at site 25 with candidate 2 and every other logical register zero, the clean store of [1]. `PA196/Startup.lean` verifies the startup, and `PA196/Nonhalting.lean` connects it to that store.

Proof of Theorem 1.1. Lemmas 7.1 and 7.2 identify halting of M from the blank tape with halting of R from the clean store. Under PA consistency, Lemma 4.1 gives nonhalting of R, so M runs forever and $\neg H$ is true in $\mathbb{N}$.

In any model of PA, the compiler turns a coded PA refutation into a halting command word. The scheduler reaches that word from candidate 2. The tape simulation joins the startup to the instruction traces, then projects the internal control codes onto the literal rows. This trace witnesses H, including when the proof code and trace length are nonstandard. Hence

$$\mathrm{PA} \vdash \mathrm{Prov}_{\mathrm{PA}}(\ulcorner \bot \urcorner) \to H, \qquad \mathrm{PA} \vdash \neg H \to \mathrm{Con}(\mathrm{PA}).$$

Gödel's second incompleteness theorem [2] excludes a PA proof of $\neg H$. If PA is sound, it does not prove the false sentence H. □

8 Lean verification

Lean checks all 392 transitions, the startup trace and the simulation contracts at all 3,812 source sites. The formal contracts use the reachable lower bound 48, which suffices for the blank-tape theorem. The finite runs at gaps 40 and 41 give additional checks.

`PA196/Arithmetic/RefutationHalting.lean` connects the arithmetic trace construction to halting. The trace sentence uses the 196 working states and the distinct halt label 196. `PA196/Independence.lean` proves the internal consistency implication and two-sided independence. `PA196/Nonhalting.lean` proves the machine theorem.

`Challenge.lean` states nonhalting and independence using an explicit PA proof calculus and the literal table. `PA196/Palomar/Solution.lean` proves those declarations and connects their definitions to Foundation and the tape model. Guarded axiom audits cover both presentations. The repository includes the table and the checked proof sources. `docs/reproduction.md` gives the finite checks, Lean build and Comparator commands.

References

[1] J. Chang. Nonhalting and PA-independence for a 209-state Turing machine. Public PA209 snapshot.

[2] K. Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik* 38 (1931), 173–198. `https://doi.org/10.1007/BF01700692`.